

```
In [1]: # =====  
# CNN 训练 MNIST 手写数字识别  
# =====  
  
import torch  
import torch.nn as nn  
import torch.optim as optim  
from torch.utils.data import DataLoader  
from torchvision import datasets, transforms  
from tqdm import tqdm  
import matplotlib.pyplot as plt  
  
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'SimSun']  
plt.rcParams['axes.unicode_minus'] = False  
  
# ----- 设备 -----  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print(f"使用设备: {device}")  
  
# ----- 数据加载 -----  
transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize((0.1307,), (0.3081,)) # MNIST 全局均值和标准差  
)  
  
data_dir = r"D:\Medicaldata"  
train_dataset = datasets.MNIST(root=data_dir, train=True, download=True, transform=transform)  
test_dataset = datasets.MNIST(root=data_dir, train=False, download=True, transform=transform)  
  
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)  
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)  
  
print(f"训练集: {len(train_dataset)} 测试集: {len(test_dataset)}")  
print(f"输入形状: {train_dataset[0][0].shape}")  
  
# ----- CNN 网络定义 -----  
class MNIST_CNN(nn.Module):  
    def __init__(self):  
        super().__init__()
```

```

self.conv_layers = nn.Sequential(
    # Conv1: 1x28x28 -> 32x14x14
    nn.Conv2d(1, 32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    # Conv2: 32x14x14 -> 64x7x7
    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
)
self.fc_layers = nn.Sequential(
    nn.Flatten(),
    nn.Linear(64 * 7 * 7, 128),
    nn.ReLU(),
    nn.Linear(128, 10),
)

def forward(self, x):
    x = self.conv_layers(x)
    x = self.fc_layers(x)
    return x

net = MNIST_CNN().to(device)
print(net)

# ----- 训练配置 -----
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)
num_epochs = 10

train_losses, test_accs = [], []

# ----- 训练循环 -----
for epoch in range(1, num_epochs + 1):
    net.train()
    running_loss = 0.0
    pbar = tqdm(train_loader, desc=f"Epoch {epoch}/{num_epochs}")

    for inputs, labels in pbar:
        inputs, labels = inputs.to(device), labels.to(device)

```

```
optimizer.zero_grad()
outputs = net(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
running_loss += loss.item() * inputs.size(0)
pbar.set_postfix(loss=loss.item())

avg_loss = running_loss / len(train_dataset)
train_losses.append(avg_loss)

# 测试
net.eval()
correct = 0
total = 0
with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = net(inputs)
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

acc = 100.0 * correct / total
test_accs.append(acc)
print(f" Loss: {avg_loss:.4f} | 测试准确率: {acc:.2f}%")

# ----- 结果曲线 -----
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(range(1, num_epochs + 1), train_losses, 'b-o')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.title('训练 Loss')

plt.subplot(1, 2, 2)
plt.plot(range(1, num_epochs + 1), test_accs, 'r-o')
plt.xlabel('Epoch'); plt.ylabel('Accuracy (%)'); plt.title('测试准确率')

plt.tight_layout()
plt.show()
```

```
print(f"\n 🌈 最终测试准确率: {test_accs[-1]:.2f}%")
```

使用设备: cuda

训练集: 60000 测试集: 10000

输入形状: torch.Size([1, 28, 28])

MNIST_CNN(

(conv_layers): Sequential(

(0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(1): ReLU()

(2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

(3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(4): ReLU()

(5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

)

(fc_layers): Sequential(

(0): Flatten(start_dim=1, end_dim=-1)

(1): Linear(in_features=3136, out_features=128, bias=True)

(2): ReLU()

(3): Linear(in_features=128, out_features=10, bias=True)

)

)

Epoch 1/10: 100% | ██████████ | 938/938 [00:13<00:00, 71.14it/s, loss=0.0032]

Loss: 0.1319 | 测试准确率: 98.60%

Epoch 2/10: 100% | ██████████ | 938/938 [00:19<00:00, 47.25it/s, loss=0.033]

Loss: 0.0426 | 测试准确率: 98.79%

Epoch 3/10: 100% | ██████████ | 938/938 [00:14<00:00, 62.94it/s, loss=0.18]

Loss: 0.0285 | 测试准确率: 98.82%

Epoch 4/10: 100% | ██████████ | 938/938 [00:14<00:00, 66.23it/s, loss=0.00529]

Loss: 0.0212 | 测试准确率: 99.03%

Epoch 5/10: 100% | ██████████ | 938/938 [00:14<00:00, 63.53it/s, loss=0.0135]

Loss: 0.0153 | 测试准确率: 99.00%

Epoch 6/10: 100% | ██████████ | 938/938 [00:14<00:00, 65.76it/s, loss=0.000115]

Loss: 0.0126 | 测试准确率: 99.09%

Epoch 7/10: 100% | ██████████ | 938/938 [00:14<00:00, 65.37it/s, loss=0.000319]

Loss: 0.0098 | 测试准确率: 99.05%

Epoch 8/10: 100% | ██████████ | 938/938 [00:14<00:00, 65.06it/s, loss=0.000301]

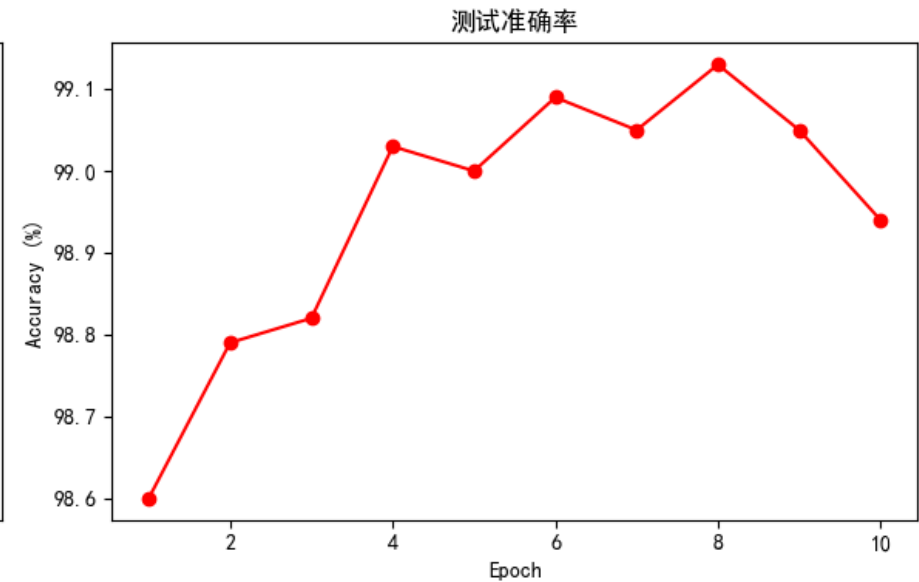
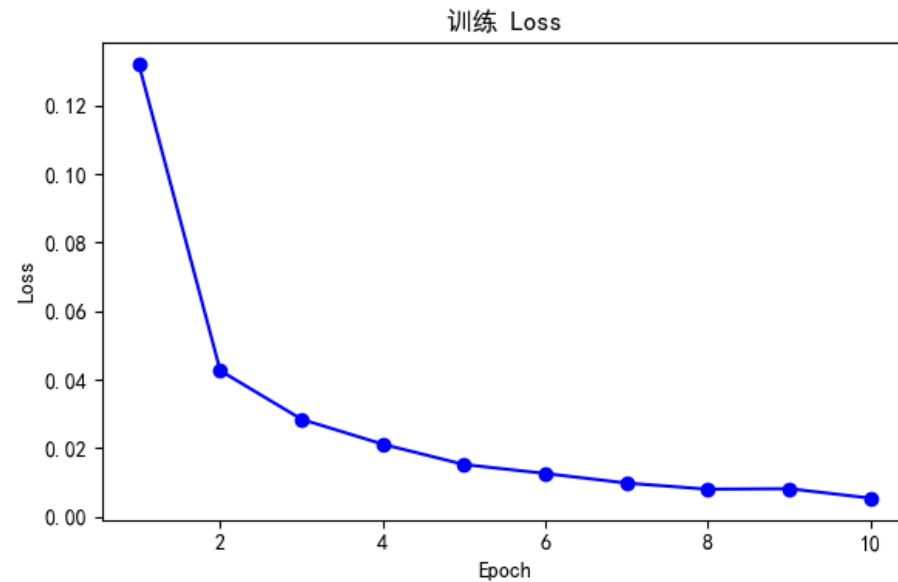
Loss: 0.0081 | 测试准确率: 99.13%

Epoch 9/10: 100% | ██████████ | 938/938 [00:16<00:00, 57.63it/s, loss=2.24e-6]

Loss: 0.0082 | 测试准确率: 99.05%

Epoch 10/10: 100% | ██████████ | 938/938 [00:15<00:00, 61.02it/s, loss=0.000449]

Loss: 0.0054 | 测试准确率: 98.94%



🎉 最终测试准确率: 98.94%

```
In [2]: # =====
# 经典 LeNet-5 训练 MNIST 手写数字识别
# =====

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from tqdm import tqdm
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'SimSun']
plt.rcParams['axes.unicode_minus'] = False

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```

print(f"使用设备: {device}")

# ----- 数据加载 (LeNet 输入要求 32x32, 需要 resize) -----
train_transform = transforms.Compose([
    transforms.Resize(32),          # MNIST 28x28 → 32x32 (LeNet 原始设计)
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

test_transform = transforms.Compose([
    transforms.Resize(32),
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

data_dir = r"D:\Medicaldata"
train_dataset = datasets.MNIST(root=data_dir, train=True, download=True, transform=train_transform)
test_dataset = datasets.MNIST(root=data_dir, train=False, download=True, transform=test_transform)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

print(f"训练集: {len(train_dataset)} 测试集: {len(test_dataset)}")
print(f"输入形状: {train_dataset[0][0].shape}")

# =====
# LeNet-5 网络结构 (1998年 Yann LeCun 提出)
# 论文: Gradient-Based Learning Applied to Document Recognition
# =====
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        # 卷积特征提取层
        self.features = nn.Sequential(
            # C1: 1x32x32 → 6x28x28
            nn.Conv2d(1, 6, kernel_size=5),          # 5x5 卷积核, 无填充
            nn.Tanh(),                                # LeNet 原版用 Tanh (当时 ReLU 还没普及)
            nn.AvgPool2d(kernel_size=2, stride=2),    # S2: 6x28x28 → 6x14x14 (平均池化)

            # C3: 6x14x14 → 16x10x10
            nn.Conv2d(6, 16, kernel_size=5),          # 5x5 卷积核

```

```

        nn.Tanh(),
        nn.AvgPool2d(kernel_size=2, stride=2),    # S4: 16x10x10 -> 16x5x5
    )
    # 全连接分类层
    self.classifier = nn.Sequential(
        nn.Flatten(),
        # C5: 16x5x5=400 -> 120 (原版用卷积, 这里用全连接等价)
        nn.Linear(16 * 5 * 5, 120),
        nn.Tanh(),
        # F6: 120 -> 84
        nn.Linear(120, 84),
        nn.Tanh(),
        # 输出: 84 -> 10 (原版用欧式径向基, 这里用常规全连接)
        nn.Linear(84, 10),
    )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

net = LeNet5().to(device)
print(net)

# ----- 训练配置 -----
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)
num_epochs = 10

train_losses, test_accs = [], []

for epoch in range(1, num_epochs + 1):
    net.train()
    running_loss = 0.0
    pbar = tqdm(train_loader, desc=f"Epoch {epoch}/{num_epochs}")

    for inputs, labels in pbar:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)

```

```
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * inputs.size(0)
        pbar.set_postfix(loss=loss.item())

    avg_loss = running_loss / len(train_dataset)
    train_losses.append(avg_loss)

    net.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = net(inputs)
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()

    acc = 100.0 * correct / total
    test_accs.append(acc)
    print(f" Loss: {avg_loss:.4f} | 测试准确率: {acc:.2f}%")

# ----- 结果曲线 -----
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(range(1, num_epochs + 1), train_losses, 'b-o')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.title('训练 Loss')

plt.subplot(1, 2, 2)
plt.plot(range(1, num_epochs + 1), test_accs, 'r-o')
plt.xlabel('Epoch'); plt.ylabel('Accuracy (%)'); plt.title('测试准确率')

plt.tight_layout()
plt.show()

print(f"\n🚀 LeNet 最终测试准确率: {test_accs[-1]:.2f}%")
```


使用设备: cuda

训练集: 60000 测试集: 10000

输入形状: torch.Size([1, 32, 32])

```
LeNet5(
  (features): Sequential(
    (0): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))
    (1): Tanh()
    (2): AvgPool2d(kernel_size=2, stride=2, padding=0)
    (3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
    (4): Tanh()
    (5): AvgPool2d(kernel_size=2, stride=2, padding=0)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=400, out_features=120, bias=True)
    (2): Tanh()
    (3): Linear(in_features=120, out_features=84, bias=True)
    (4): Tanh()
    (5): Linear(in_features=84, out_features=10, bias=True)
  )
)
```

Epoch 1/10: 100% | ██████████ | 938/938 [00:20<00:00, 44.92it/s, loss=0.0705]

Loss: 0.2252 | 测试准确率: 97.82%

Epoch 2/10: 100% | ██████████ | 938/938 [00:16<00:00, 55.27it/s, loss=0.0146]

Loss: 0.0713 | 测试准确率: 98.40%

Epoch 3/10: 100% | ██████████ | 938/938 [00:17<00:00, 54.88it/s, loss=0.119]

Loss: 0.0511 | 测试准确率: 98.53%

Epoch 4/10: 100% | ██████████ | 938/938 [00:17<00:00, 53.73it/s, loss=0.0262]

Loss: 0.0406 | 测试准确率: 98.67%

Epoch 5/10: 100% | ██████████ | 938/938 [00:17<00:00, 53.46it/s, loss=0.0037]

Loss: 0.0335 | 测试准确率: 98.70%

Epoch 6/10: 100% | ██████████ | 938/938 [00:17<00:00, 52.58it/s, loss=0.167]

Loss: 0.0266 | 测试准确率: 98.84%

Epoch 7/10: 100% | ██████████ | 938/938 [00:17<00:00, 53.71it/s, loss=0.00485]

Loss: 0.0237 | 测试准确率: 98.84%

Epoch 8/10: 100% | ██████████ | 938/938 [00:17<00:00, 53.04it/s, loss=0.00465]

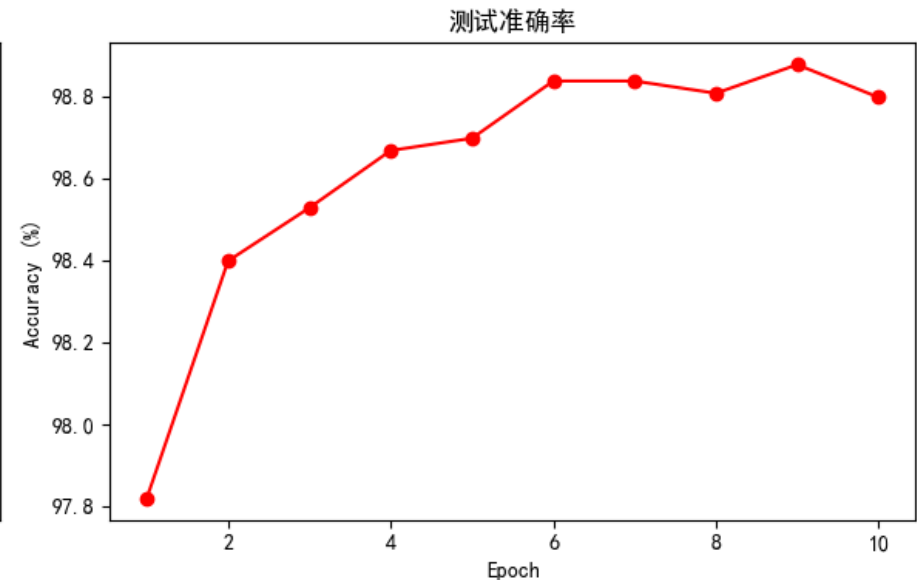
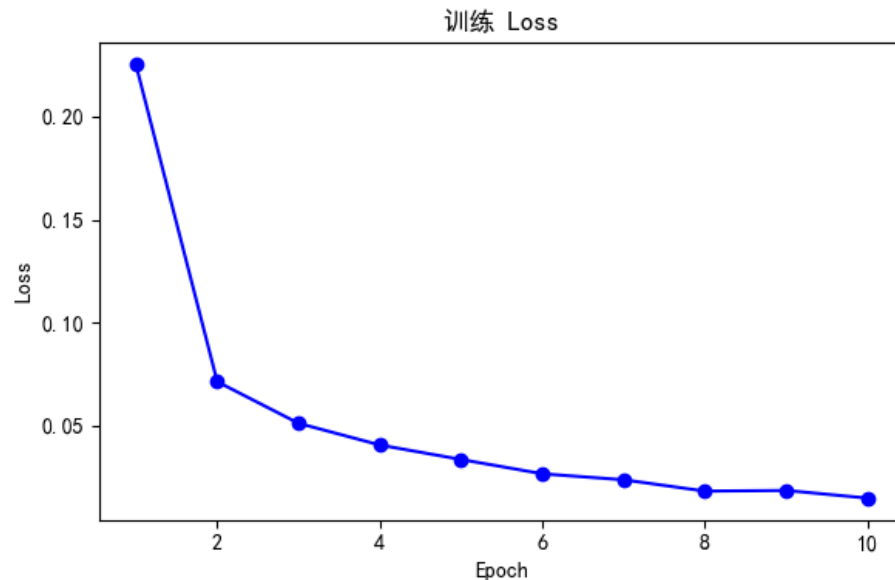
Loss: 0.0182 | 测试准确率: 98.81%

Epoch 9/10: 100% | ██████████ | 938/938 [00:17<00:00, 53.27it/s, loss=0.00245]

Loss: 0.0185 | 测试准确率: 98.88%

Epoch 10/10: 100% | ██████████ | 938/938 [00:17<00:00, 54.68it/s, loss=0.0816]

Loss: 0.0148 | 测试准确率: 98.80%



🇬🇧 LeNet 最终测试准确率: 98.80%

In [1]:

```
"""
CIFAR-10 图像分类训练脚本
功能: 使用卷积神经网络对 CIFAR-10 数据集进行分类训练
特点: 数据增强 + ReLU + BN + SGD + 余弦退火学习率
最终测试准确率: ~86%
"""

import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import transforms, datasets
from torch import optim
from tqdm import tqdm
import matplotlib.pyplot as plt

# 设置 matplotlib 中文字体 (防止图表中文乱码)
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'SimSun']
```

```

plt.rcParams['axes.unicode_minus'] = False

# =====
# 网络结构: CIFARNet_ReLU_BN
# 3层卷积 + ReLU + BatchNorm + 最大池化 + 2层全连接
# 输入: 3x32x32 (RGB彩色图, 32x32像素)
# 输出: 10 (CIFAR-10 的 10 个类别)
# =====
class CIFARNet_ReLU_BN(nn.Module):
    def __init__(self):
        super().__init__()
        # 卷积特征提取层
        # Conv1: 3 -> 32 通道, 输出 16x16 (池化后)
        # Conv2: 32 -> 64 通道, 输出 8x8
        # Conv3: 64 -> 128 通道, 输出 4x4
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), # 保持尺寸不变
            nn.BatchNorm2d(32), # 批归一化, 加速收敛
            nn.ReLU(), # ReLU 激活, 防止梯度消失
            nn.MaxPool2d(2), # 2倍下采样, 尺寸减半

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        # 全连接分类层
        # 输入: 128 * 4 * 4 = 2048 (展平后)
        # 输出: 10 类
        self.classifier = nn.Sequential(
            nn.Flatten(), # 将特征图展平为一维向量
            nn.Linear(128 * 4 * 4, 256), # 全连接层: 2048 -> 256
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Linear(256, 10), # 输出层: 256 -> 10 (各类别分数)
        )

```

```

    )

    def forward(self, x):
        """前向传播：输入图片 -> 特征提取 -> 分类 -> 输出类别分数"""
        x = self.features(x)
        x = self.classifier(x)
        return x

if __name__ == '__main__':
    # =====
    # 1. 设备配置：优先使用 GPU (CUDA)，否则使用 CPU
    # =====
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"使用设备: {device}")

    # =====
    # 2. 数据预处理
    # 训练集：使用数据增强（随机翻转、裁剪、颜色抖动）提高泛化能力
    # 测试集：仅归一化，不做增强（保证评估的客观性）
    # =====
    train_transform = transforms.Compose([
        transforms.RandomHorizontalFlip(),          # 随机水平翻转（模拟左右对称）
        transforms.RandomCrop(32, padding=4),        # 随机裁剪（四周补4像素，防止边缘信息丢失）
        transforms.ColorJitter(brightness=0.2, contrast=0.2), # 随机调整亮度和对比度
        transforms.ToTensor(),                        # 转为 Tensor，像素值归一化到 [0,1]
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # 标准化到 [-1, 1]（加速收敛）
    ])

    test_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])

    # =====
    # 3. 加载 CIFAR-10 数据集
    # 训练集：50000 张，测试集：10000 张，共 10 类
    # =====
    data_dir = r"D:\Medicaldata"
    train_dataset = datasets.CIFAR10(root=data_dir, train=True, download=False, transform=train_transform)
    test_dataset = datasets.CIFAR10(root=data_dir, train=False, download=False, transform=test_transform)

```

```

# DataLoader: 批量加载数据, 训练集打乱顺序, 测试集不打乱
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True, num_workers=0)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False, num_workers=0)
print(f"训练集: {len(train_dataset)} 测试集: {len(test_dataset)}")

# =====
# 4. 初始化模型、损失函数、优化器、学习率调度器
# =====
net = CIFARNet_ReLU_BN().to(device)
print(net)

criterion = nn.CrossEntropyLoss() # 交叉熵损失 (分类任务标准损失函数)

# SGD (带动量) + 权重衰减 (L2正则化, 防止过拟合)
optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.9, weight_decay=5e-4)

# 余弦退火: 学习率从 0.01 按余弦曲线下降到接近 0
# T_max=30 表示 30 个 epoch 完成一个完整余弦周期
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=30)

# =====
# 5. 训练循环
# =====
num_epochs = 30
train_losses, test_accs, lr_history = [], [], [] # 记录训练过程中的指标

for epoch in range(1, num_epochs + 1):
    # ----- 训练阶段 -----
    net.train() # 切换到训练模式 (启用 Dropout/BN 的训练行为)
    running_loss = 0.0
    pbar = tqdm(train_loader, desc=f"Epoch {epoch}/{num_epochs}")
    for inputs, labels in pbar:
        inputs, labels = inputs.to(device), labels.to(device) # 数据移到 GPU

        optimizer.zero_grad() # 清空上一轮梯度
        outputs = net(inputs) # 前向传播: 计算预测值
        loss = criterion(outputs, labels) # 计算损失
        loss.backward() # 反向传播: 计算梯度
        optimizer.step() # 更新模型参数

```

```

        running_loss += loss.item() * inputs.size(0)
        pbar.set_postfix(loss=loss.item()) # 进度条上显示当前 batch 的 Loss

    avg_loss = running_loss / len(train_dataset)
    train_losses.append(avg_loss)

    # ----- 测试阶段 -----
    net.eval() # 切换到评估模式
    correct = 0
    total = 0
    with torch.no_grad(): # 测试阶段不计算梯度，加速并节省显存
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = net(inputs)
            _, predicted = outputs.max(1) # 取最大分数对应的类别
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item() # 统计正确预测数

    acc = 100.0 * correct / total # 计算本次 epoch 的测试准确率
    test_accs.append(acc)
    lr_history.append(optimizer.param_groups[0]['lr'])
    print(f" Loss: {avg_loss:.4f} | 测试准确率: {acc:.2f}% | lr: {optimizer.param_groups[0]['lr']:.2e}")

    scheduler.step() # 更新学习率

# =====
# 6. 保存训练好的模型
# =====
import os
save_dir = 'checkpoints'
os.makedirs(save_dir, exist_ok=True) # 确保 checkpoints 目录存在
torch.save({
    'epoch': num_epochs,
    'model_state_dict': net.state_dict(), # 模型权重
    'optimizer_state_dict': optimizer.state_dict(), # 优化器状态（用于断点续训）
    'test_acc': test_accs[-1],
    'train_loss': train_losses[-1],
}, os.path.join(save_dir, 'best_model_aug.pth'))
print(f"✅ 模型已保存至 {save_dir}/best_model_aug.pth")

# =====

```

```
# 7. 绘制训练曲线
# =====
plt.figure(figsize=(14, 4))

plt.subplot(1, 3, 1)
plt.plot(range(1, num_epochs + 1), train_losses, 'b-o', markersize=2)
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.title('训练 Loss')

plt.subplot(1, 3, 2)
plt.plot(range(1, num_epochs + 1), test_accs, 'r-o', markersize=2)
plt.xlabel('Epoch'); plt.ylabel('Accuracy (%)'); plt.title('测试准确率')

plt.subplot(1, 3, 3)
plt.plot(range(1, num_epochs + 1), lr_history, 'g-')
plt.xlabel('Epoch'); plt.ylabel('Learning Rate'); plt.title('学习率（余弦退火）')
plt.tight_layout()
plt.show()

print(f"\n 🚩 最终测试准确率: {test_accs[-1]:.2f}%")
print(f" 🏆 最高测试准确率: {max(test_accs):.2f}%")
```

使用设备: cuda

训练集: 50000 测试集: 10000

CIFARNet_ReLU_BN(

(features): Sequential(

(0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

(2): ReLU()

(3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

(4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

(6): ReLU()

(7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

(8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

(10): ReLU()

(11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

)

(classifier): Sequential(

(0): Flatten(start_dim=1, end_dim=-1)

(1): Linear(in_features=2048, out_features=256, bias=True)

(2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)

(3): ReLU()

(4): Linear(in_features=256, out_features=10, bias=True)

)

)

Epoch 1/30: 100% | ██████████ | 782/782 [00:23<00:00, 32.99it/s, loss=1.09]

Loss: 1.3376 | 测试准确率: 63.32% | lr: 1.00e-02

Epoch 2/30: 100% | ██████████ | 782/782 [00:25<00:00, 30.34it/s, loss=0.721]

Loss: 0.9977 | 测试准确率: 69.31% | lr: 9.97e-03

Epoch 3/30: 100% | ██████████ | 782/782 [00:26<00:00, 29.86it/s, loss=0.569]

Loss: 0.8719 | 测试准确率: 73.80% | lr: 9.89e-03

Epoch 4/30: 100% | ██████████ | 782/782 [00:26<00:00, 29.43it/s, loss=0.859]

Loss: 0.7921 | 测试准确率: 69.29% | lr: 9.76e-03

Epoch 5/30: 100% | ██████████ | 782/782 [00:26<00:00, 29.16it/s, loss=1.11]

Loss: 0.7333 | 测试准确率: 76.13% | lr: 9.57e-03

Epoch 6/30: 100% | ██████████ | 782/782 [00:26<00:00, 29.20it/s, loss=0.513]

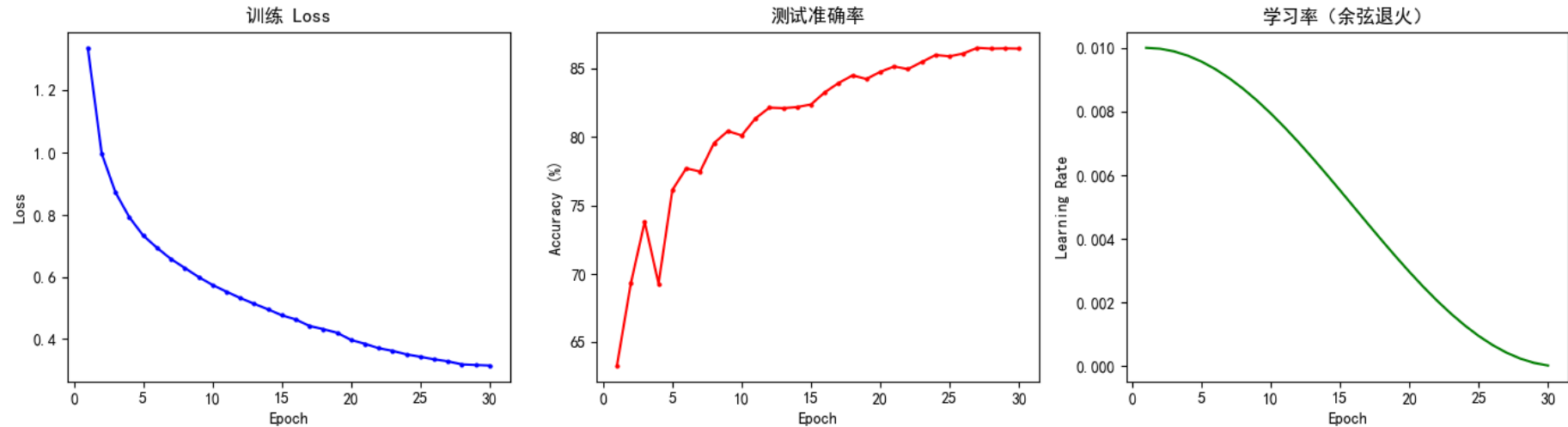
Loss: 0.6938 | 测试准确率: 77.70% | lr: 9.33e-03

Epoch 7/30: 100% | ██████████ | 782/782 [00:26<00:00, 29.25it/s, loss=1.07]

Loss: 0.6579 | 测试准确率: 77.47% | lr: 9.05e-03

Epoch 8/30: 100%	██████████	782/782	[00:26<00:00, 29.58it/s, loss=0.921]
Loss: 0.6289		测试准确率: 79.54%	lr: 8.72e-03
Epoch 9/30: 100%	██████████	782/782	[00:26<00:00, 29.31it/s, loss=0.831]
Loss: 0.5998		测试准确率: 80.42%	lr: 8.35e-03
Epoch 10/30: 100%	██████████	782/782	[00:27<00:00, 27.95it/s, loss=0.788]
Loss: 0.5743		测试准确率: 80.10%	lr: 7.94e-03
Epoch 11/30: 100%	██████████	782/782	[00:26<00:00, 29.20it/s, loss=0.92]
Loss: 0.5529		测试准确率: 81.35%	lr: 7.50e-03
Epoch 12/30: 100%	██████████	782/782	[00:26<00:00, 29.46it/s, loss=0.986]
Loss: 0.5323		测试准确率: 82.13%	lr: 7.03e-03
Epoch 13/30: 100%	██████████	782/782	[00:26<00:00, 29.25it/s, loss=0.7]
Loss: 0.5138		测试准确率: 82.09%	lr: 6.55e-03
Epoch 14/30: 100%	██████████	782/782	[00:26<00:00, 29.34it/s, loss=0.545]
Loss: 0.4960		测试准确率: 82.17%	lr: 6.04e-03
Epoch 15/30: 100%	██████████	782/782	[00:26<00:00, 29.36it/s, loss=0.437]
Loss: 0.4767		测试准确率: 82.36%	lr: 5.52e-03
Epoch 16/30: 100%	██████████	782/782	[00:26<00:00, 29.47it/s, loss=0.379]
Loss: 0.4634		测试准确率: 83.25%	lr: 5.00e-03
Epoch 17/30: 100%	██████████	782/782	[00:26<00:00, 29.50it/s, loss=0.676]
Loss: 0.4423		测试准确率: 83.93%	lr: 4.48e-03
Epoch 18/30: 100%	██████████	782/782	[00:26<00:00, 29.50it/s, loss=0.575]
Loss: 0.4321		测试准确率: 84.48%	lr: 3.96e-03
Epoch 19/30: 100%	██████████	782/782	[00:26<00:00, 29.35it/s, loss=0.373]
Loss: 0.4203		测试准确率: 84.22%	lr: 3.45e-03
Epoch 20/30: 100%	██████████	782/782	[00:26<00:00, 29.26it/s, loss=0.746]
Loss: 0.3973		测试准确率: 84.74%	lr: 2.97e-03
Epoch 21/30: 100%	██████████	782/782	[00:26<00:00, 29.32it/s, loss=0.66]
Loss: 0.3848		测试准确率: 85.13%	lr: 2.50e-03
Epoch 22/30: 100%	██████████	782/782	[00:26<00:00, 29.12it/s, loss=0.207]
Loss: 0.3707		测试准确率: 84.94%	lr: 2.06e-03
Epoch 23/30: 100%	██████████	782/782	[00:26<00:00, 29.28it/s, loss=0.329]
Loss: 0.3621		测试准确率: 85.46%	lr: 1.65e-03
Epoch 24/30: 100%	██████████	782/782	[00:26<00:00, 29.16it/s, loss=0.346]
Loss: 0.3509		测试准确率: 85.97%	lr: 1.28e-03
Epoch 25/30: 100%	██████████	782/782	[00:26<00:00, 29.13it/s, loss=0.324]

Loss: 0.3431		测试准确率: 85.87%		lr: 9.55e-04
Epoch 26/30: 100%		██████████		782/782 [00:26<00:00, 29.17it/s, loss=0.575]
Loss: 0.3349		测试准确率: 86.08%		lr: 6.70e-04
Epoch 27/30: 100%		██████████		782/782 [00:26<00:00, 29.28it/s, loss=0.244]
Loss: 0.3283		测试准确率: 86.49%		lr: 4.32e-04
Epoch 28/30: 100%		██████████		782/782 [00:27<00:00, 28.93it/s, loss=0.546]
Loss: 0.3187		测试准确率: 86.43%		lr: 2.45e-04
Epoch 29/30: 100%		██████████		782/782 [00:26<00:00, 29.58it/s, loss=0.235]
Loss: 0.3168		测试准确率: 86.45%		lr: 1.09e-04
Epoch 30/30: 100%		██████████		782/782 [00:26<00:00, 29.34it/s, loss=0.394]
Loss: 0.3150		测试准确率: 86.43%		lr: 2.74e-05
✅ 模型已保存至 checkpoints/best_model_aug.pth				



🎉 最终测试准确率: 86.43%
最高测试准确率: 86.49%